

Semesterüberblick

1

Skript: auf Moodle

Bonuspunkte: 2 Punkte pro Mini Quiz (alle 2 Wochen) beginnt nächste Woche!

2 Punkte pro Codeexpert-Aufgabe (alle 2 Wochen)

2 Punkte pro Theorie-Aufgabe mit Peergrading (alle 2 Wochen)

maximal 0,25 Notenerhöhung

josia-heger.com: hier findet ihr meine Notizen

WhatsApp Gruppe für Fragen

Algorithm Lab

Allgemeines: A&W ist meiner Meinung nach euer schwierigstes Fach

DDCA: macht die Labs gut, damit ihr 30% der Prüfungspunkte schon bekommt

lasst euch nicht von 200 Slides pro Vorlesung einschüchtern

PProg: ab ~Woche 7 wird es schwieriger

Prüfung eher leicht (jedes Jahr sehr ähnliche Aufgaben)

Analysis: Übt viele Aufgaben, Beweise sind ziemlich egal

(vielleicht werdet ihr DiskMat vermissen ..)

Osterferien

Sommer-Lernphase ist doppelt so lang wie im Winter

Zusätzliche Fächer?

Vorlesungen nachschauen?

Wieso A&D und A&W: viele Beispiel-Algorithmen für z.B. suchen, sortieren, ...

Das Hauptziel ist NICHT, dass ihr suchen/sortieren könnt,
sondern dass ihr verschiedene Strategien zum lösen von Problemen
lernt, z.B.

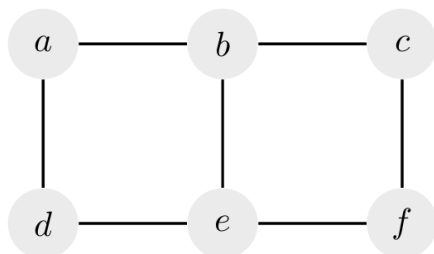
- Grosses Problem in viele Teilprobleme aufteilen (DP)
- kleine lokale Verbesserungen (BubbleSort)
- Problem halbieren, Lösungen zusammenführen (MergeSort)
- Zufall (QuickSort)
- Hilfs-Datenstruktur erstellen (Heap-Sort, Dijkstra)
- Rekursion (DFS/BFS)
- Greedy Algorithmen (Prim/Kruskal)
- Probleme umformulieren (Graphen-Textaufgaben)

in A&W kommen viele neue Werkzeuge dazu!

Warm-up Aufgaben

Exercise 0.1 – Paths, Walks, Cycles

Consider the following graph $G = (V, E)$.



1. Which paths of length 4 (i.e. consisting of 4 edges) are there from a to e ?

only one path (a, b, c, f, e)

2. Which walks of length 4 (i.e. consisting of 4 edges) are there from a to e ?

first two edges	number of possible choices for last two edges
a, b, a	2
a, b, c	2
a, b, e	3
a, d, a	2
a, d, e	3

↳ sum = 12

3. Which cycles are there in G ?

(a, b, e, d, a) , (b, c, f, e, b) , (a, b, c, f, e, d, a)

4. How many closed walks are there in G ?

Infinitely many, e.g.: (a, b, a) , (a, b, a, b, a) , (a, b, a, b, a, b, a) ..

Exercise 0.2 – Asymptotic Growth

- (a) (Simpler) Sort the following functions asymptotically, i.e., according to O-notation. Here, $\log n$ denotes the logarithm to base 2 of n , and $\ln n$ denotes the natural logarithm of n . In which cases do you have asymptotic equality $\Theta(\cdot)$?

$$n, \quad 0.01n^2, \quad e^n, \quad \log n, \quad 2^{32}, \quad 2^n, \quad n + \sqrt{n},$$

$\leq$ bezieht sich hier auf die O-Notation und $=$ auf die Θ -Notation

$$2^{32} \leq \log n \leq \underbrace{n = n + \sqrt{n}}_{\substack{\text{weil } n \leq n + \sqrt{n} \leq 2n \\ \text{und } n \in \Theta(n) \text{ und } 2n \in \Theta(n)}} \leq 0.01n^2 \leq 2^n \leq \underbrace{e^n}_{e \approx 2.718}$$

- (b) (Harder) Additionally, insert the following functions into your sequence.

$$\ln n, \quad \frac{n}{\log n}, \quad e^{\sqrt{\log n}}, \quad \log(n^2), \quad n^{1/4}, \quad n!$$

$$2^{32} \leq \underbrace{\log n = \ln n = \log(n^2)}_{\substack{\text{alle Logarithmen wachsen} \\ \text{asymptotisch gleich schnell,} \\ \log(n^2) = 2 \log(n)}} \leq \dots \text{ wie weiter? } \dots \leq n!$$

alle Logarithmen wachsen
asymptotisch gleich schnell,
 $\log(n^2) = 2 \log(n)$

$$* \text{ weil } \frac{e^{\sqrt{\log n}}}{n} = \frac{e^{\sqrt{\log n}}}{e^{\ln(n)}} = e^{\overbrace{\sqrt{\log n} - \ln(n)}^{\rightarrow -\infty}} \rightarrow 0$$

$$\left. \begin{array}{l} \text{Feststellung: } \frac{n}{\log n} \leq n \\ \frac{n}{e^{\sqrt{\log n}}} \leq n \\ n^{\frac{1}{4}} = \sqrt[4]{n} \leq n \end{array} \right\} \text{ alle drei langsamer als } n$$

$$\text{Grenzwerte bestimmen: } \lim_{n \rightarrow \infty} \frac{n^{\frac{1}{4}}}{e^{\sqrt{\log n}}} = \lim_{n \rightarrow \infty} \frac{e^{\ln(n^{\frac{1}{4}})}}{e^{\sqrt{\log n}}} = \lim_{n \rightarrow \infty} \frac{e^{\frac{1}{4} \cdot \ln n}}{e^{\sqrt{\log n}}}$$

$e^{\ln}$ Trick

log Gesetz

Potenzgesetz

$$= \lim_{n \rightarrow \infty} e^{\overbrace{\frac{1}{4} \ln(n) - \sqrt{\log n}}^{\text{Exponent} \rightarrow \infty}} = \infty$$

$$\text{weil } \lim_{n \rightarrow \infty} \frac{1}{4} \ln(n) - \sqrt{\log n} = \lim_{n \rightarrow \infty} \frac{1}{4} \ln(n) - \sqrt{\ln(n)} = \lim_{x \rightarrow \infty} \frac{1}{4} x - \sqrt{x} = \infty$$

logs wachsen
gleich schnell

substitution $x = \ln(n)$

$$\Rightarrow e^{\sqrt{\log n}} \leq O(n^{\frac{1}{4}})$$

$$\begin{aligned} \text{Nächster Grenzwert: } \lim_{n \rightarrow \infty} \frac{n^{\frac{1}{4}}}{\log n} &= \lim_{n \rightarrow \infty} \frac{n^{\frac{1}{4}} \log n}{n} = \lim_{n \rightarrow \infty} \frac{\cancel{n^{\frac{1}{4}}} \log n}{n^{\frac{3}{4}}} \stackrel{\text{L'Hôp.}}{=} \lim_{n \rightarrow \infty} \frac{\frac{1}{n}}{\frac{3}{4} \cdot n^{-\frac{1}{4}}} \\ &\stackrel{\text{mit Kehrwert multiplizieren}}{=} \lim_{n \rightarrow \infty} \frac{1}{\frac{3}{4} \cdot n^{-\frac{1}{4}}} = \lim_{n \rightarrow \infty} \frac{1}{\frac{3}{4} \cdot n^{\frac{3}{4}}} = 0 \end{aligned}$$

$$\Rightarrow n^{\frac{1}{4}} \leq O\left(\frac{n}{\log n}\right)$$

$$\text{Nächster Grenzwert: } \lim_{n \rightarrow \infty} \frac{e^{\sqrt{\log n}}}{\log n} = \lim_{x \rightarrow \infty} \frac{e^{\sqrt{x}}}{x} = \lim_{x \rightarrow \infty} \frac{e^{\sqrt{x}}}{e^{\ln(x)}} = \lim_{x \rightarrow \infty} e^{\overbrace{\sqrt{x} - \ln(x)}^{\text{Exponent} \rightarrow \infty}} = \infty$$

weil polynomiell schneller

wächst als logarithmisch

$$\Rightarrow \log n \leq O(e^{\sqrt{\log n}})$$

Insgesamt:

$$2^{32} \leq \log n = \ln n = \log(n^2) \leq e^{\sqrt{\log n}} \leq n^{\frac{1}{4}} \leq \frac{n}{\log n} \leq n = n + \sqrt{n} \leq 0.01 n^2 \leq 2^n \leq e^n \leq n!$$

Exercise 0.3 – Induction

(a) Show that for all $n \in \mathbb{N}$:

$$\frac{1}{1 \cdot 2} + \frac{1}{2 \cdot 3} + \frac{1}{3 \cdot 4} + \dots + \frac{1}{n \cdot (n+1)} = \frac{n}{n+1}.$$

Base Case: $k=1 \Rightarrow \frac{1}{1 \cdot 2} = \frac{1}{1 \cdot (1+1)}$ holds

I.H. Assume that the statement holds for some $k \in \mathbb{N}$, $k \geq 1$

$$\begin{aligned} \text{I.S.: } \frac{1}{1 \cdot 2} + \frac{1}{2 \cdot 3} + \dots + \frac{1}{k(k+1)} + \frac{1}{(k+1)(k+2)} & \stackrel{\text{I.H.}}{=} \frac{k}{k+1} + \frac{1}{(k+1)(k+2)} \\ &= \frac{k \cdot (k+2)}{(k+1)(k+2)} + \frac{1}{(k+1)(k+2)} \\ &= \frac{k^2 + 2k + 1}{(k+1)(k+2)} \\ &= \frac{(k+1)^2}{(k+1)(k+2)} \quad \leftarrow \text{Binomische Formel} \\ &= \frac{k+1}{k+2} \\ &= \frac{k+1}{(k+1) + 1} \quad \square \end{aligned}$$

(b) Prove *Bernoulli's inequality*: For all natural numbers $n \geq 1$ and all $h \in \mathbb{R}$ with $h \geq -1$, the following holds:

$$1 + nh \leq (1 + h)^n.$$

let $h \geq -1$ be arbitrary

Base Case: $k=1 \Rightarrow 1 + 1 \cdot h = 1 + h \leq (1 + h)^1$ holds

I.H. Assume that the statement holds for some $k \in \mathbb{N}$, $k \geq 1$

$$\begin{aligned} \text{I.S.: } (1+h)^{k+1} &= \underbrace{(1+h)^k}_{\text{I.H.}} (1+h) \\ &\geq \underbrace{(1+kh)}_{\text{weil } (1+h) \geq 0} \cdot (1+h) \\ &= 1+h+kh+kh^2 \quad \leftarrow \text{ausmultiplizieren} \\ &\geq 1+h+kh \quad \leftarrow kh^2 \geq 0 \text{ weglassen} \\ &= 1+(k+1)h \quad \square \end{aligned}$$

Exercise 0.4 – A General Property of Graphs

Prove that every graph G with $n \geq 2$ vertices contains two distinct vertices $v \neq w$ such that $\deg(v) = \deg(w)$.

Hint: For a given n , what is the maximum possible degree a vertex can have?

minimum possible degree = 0

maximum possible degree = $n-1$ if one node has an edge to all other $n-1$ nodes

Case: there exists a vertex with degree 0

$\Rightarrow$ there exists no vertex with degree $n-1$

$\Rightarrow$ there are only $n-1$ possible degrees, i.e. $0, 1, \dots, n-2$, but n nodes

$\Rightarrow$ two nodes must have the same degree by the pigeonhole principle

Case: there exists no vertex with degree 0

$\Rightarrow$ there are only $n-1$ possible degrees, i.e. $1, 2, \dots, n-1$, but n nodes

$\Rightarrow$ two nodes must have the same degree by the pigeonhole principle

Exercise 0.5 – Algorithm

Describe an algorithm that solves the following problem: Given an input consisting of a graph $G = (V, E)$ with n vertices (assume the graph is given as an adjacency list), your algorithm should output “Yes” if G is a tree and “No” otherwise.

As always, a complete solution should include: a clear description of the algorithm, a proof of correctness, and a runtime analysis.

Hint: For this exercise, you may use the statement from Exercise 6 without proof.

Algorithm: `isTree(G):`

if $|E| \neq |V| - 1$ return No

mark all $v \in V$ as not visited

DFS(v) where v is an arbitrary starting node

if $\exists v \in V$ that was not visited return No

else return Yes

Correctness: by Theorem 1.6 (Skript)

Runtime: we stop counting edges as soon as we have already counted more than $|V| - 1$ edges and immediately return NO

This trick allows us to execute the first line in $O(|V|)$

DFS runs in $O(|V| + |E|) \leq O(|V| + |V| - 1) \leq O(|V|)$

Total runtime: $O(|V|)$

Exercise 0.6 – Characterization of Trees (Challenge Problem)

Show that for a graph $G = (V, E)$ with $|V| \geq 1$ vertices, the following statements are equivalent:

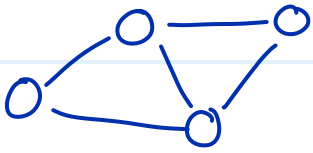
- (a) G is connected and acyclic (i.e., G is a tree).
- (b) G is connected and $|E| = |V| - 1$.
- (c) G is acyclic and $|E| = |V| - 1$.
- (d) For all $x, y \in V$: G contains exactly one x - y path.

Definitionen:

G ist k -Knoten-zusammenhängend $\Leftrightarrow$ egal welche $k-1$ Knoten man entfernt, der Graph bleibt zusammenhängend

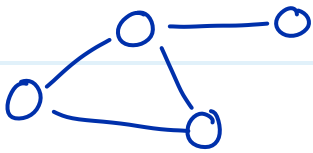
G ist k -Kanten-zusammenhängend $\Leftrightarrow$ egal welche $k-1$ Kanten man entfernt, der Graph bleibt zusammenhängend

Beispiele:



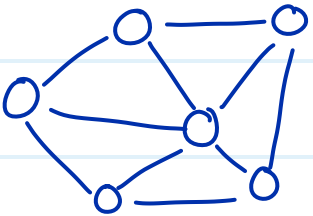
Knoten-zusammenhang = 2

Kanten-zusammenhang = 2



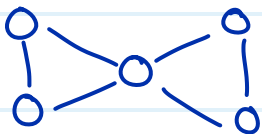
Knoten-zusammenhang = 1

Kanten-zusammenhang = 1



Knoten-zusammenhang = 3

Kanten-zusammenhang = 3



Knoten-zusammenhang = 1

Kanten-zusammenhang = 2

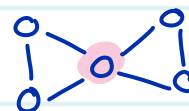
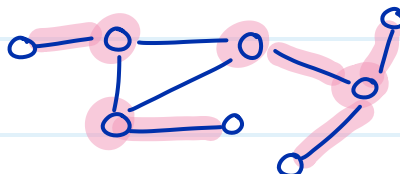
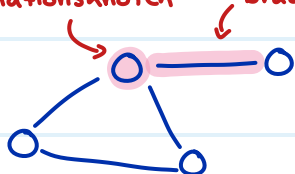
Definition:

$e \in E$ ist eine Brücke $\Leftrightarrow G$ ist zusammenhängend aber $G - e$ ist nicht zusammenhängend

$v \in V$ ist ein Artikulationsknoten $\Leftrightarrow G$ ist zusammenhängend aber $G[V \setminus \{v\}]$ ist

nicht zusammenhängend

Artikulationsknoten Brücke



(mehr Theorie dazu in der nächsten Übungsstunde)

Exercise P1.1 – Connectivity

Let $G = (V, E)$ be a two connected graph.

- (a) Let (u, v, w) be a path in G of length 2. Show that we can extend this path to a cycle, that is, show that G contains a cycle where u, v, w appear as incident vertices.

After removing v , $G[V \setminus \{v\}]$ is still connected. (by def of 2-connected)

$\Rightarrow$ there exists a Path $P = p_1, p_2, \dots, p_k$ from u to w in $G[V \setminus \{v\}]$ (def. connected)

$\Rightarrow p_1, p_2, \dots, p_k, v, u$ is a cycle in G

$\parallel$ $\parallel$
 u w

- (b) Let $e \in E$ be an edge, and $u, v \in V$ be two vertices. Show that there is a path in G from u to v passing through e .

(Hint: you may use without proof that in a 2-connected graph, every two edges e, f lie on a common cycle.)

Case: $e = \{u, v\} \Rightarrow e$ is the trivial solution

Case: $e \neq \{u, v\}$

$G(V, E \cup \{\{u, v\}\})$ is 2 connected. (adding more edges never decreases the connectivity)

$\Rightarrow e$ and $\{u, v\}$ lie on a common cycle C in $G(V, E \cup \{\{u, v\}\})$ (from hint)

$\Rightarrow$ removing the edge $\{u, v\}$ from C results in a Path from u to v passing through e

Ein König besitzt 100 Weinfässer, von denen genau eines mit einem tödlichen Gift versetzt wurde. Die Symptome treten exakt **sieben Tage** nach dem Verzehr auf. Dem König bleiben genau sieben Tage Zeit bis zum nächsten Fest, um das vergiftete Fass zu identifizieren. Wie viele Vorkoster werden **mindestens** benötigt, um das vergiftete Fass durch eine strategische Testreihe eindeutig zu bestimmen?



Vorkoster 1 trinkt aus 1-50



Vorkoster 2 trinkt aus 1-25 und 51-75

⋮

$$2^6 = 64$$

$$2^7 = 128 \Rightarrow \text{Es braucht mindestens 7 Vorkoster}$$

Quiz nächste Woche!